



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 33

Dr. Asad Mahmood

April 21, 2026

Lecture Outline

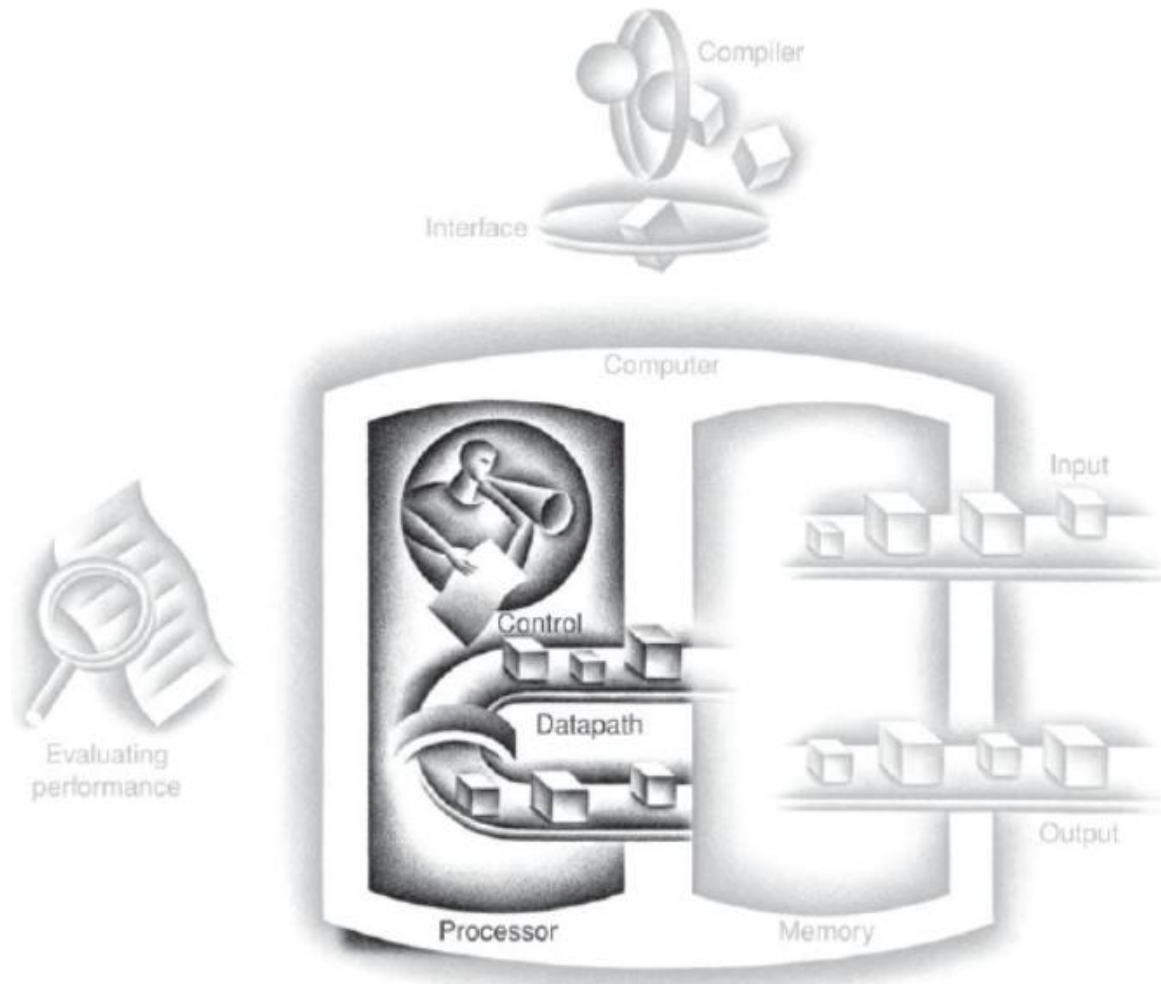
- Announcements (Assignment and Quiz 4, Prof. Qadir talk, Wed Class)
- Outline of Previous Lecture:
 - **Chapter 4 – The Processor**
 - 4.6 An Overview of Pipelining
- Outline of Today's Lecture:
 - **Chapter 4 – The Processor**
 - 4.6 An Overview of Pipelining
 - 4.7 Pipelined Datapath and Control

Chapter 4

The Processor

Chapter Theme

The Five Classic Components of a Computer



Chapter Outline

Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

Hazards

- **Hazards** are situations that prevent starting the next instruction in the next cycle
- Three types:
 1. Structural hazards
 2. Data hazards
 3. Control hazards

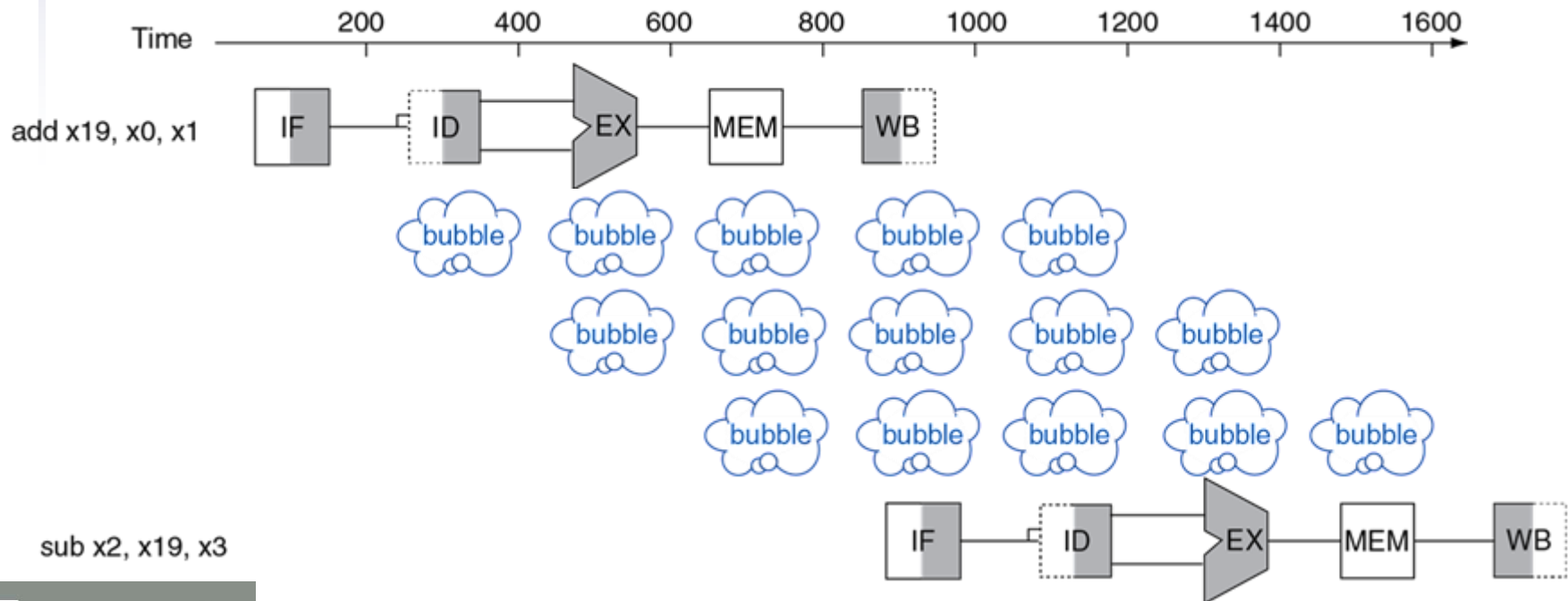
Structural Hazards

- **Conflict** for use of a resource (laundry analogy)
- In RISC-V pipeline with a single memory
 - Load/store requires data access
 - **Instruction fetch would have to *stall*** for that cycle – see previous example
 - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories/caches

Data Hazards

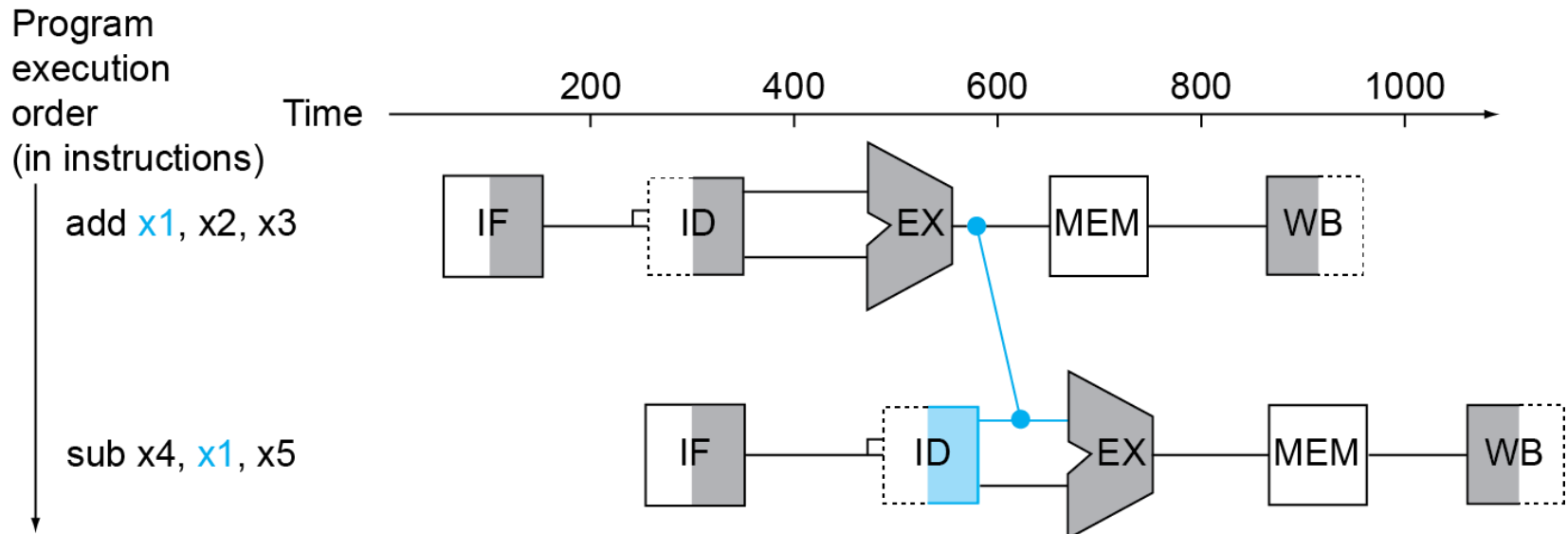
- An instruction depends on completion of data access by a previous instruction

- add **x19**, x0, x1
sub x2, **x19**, x3



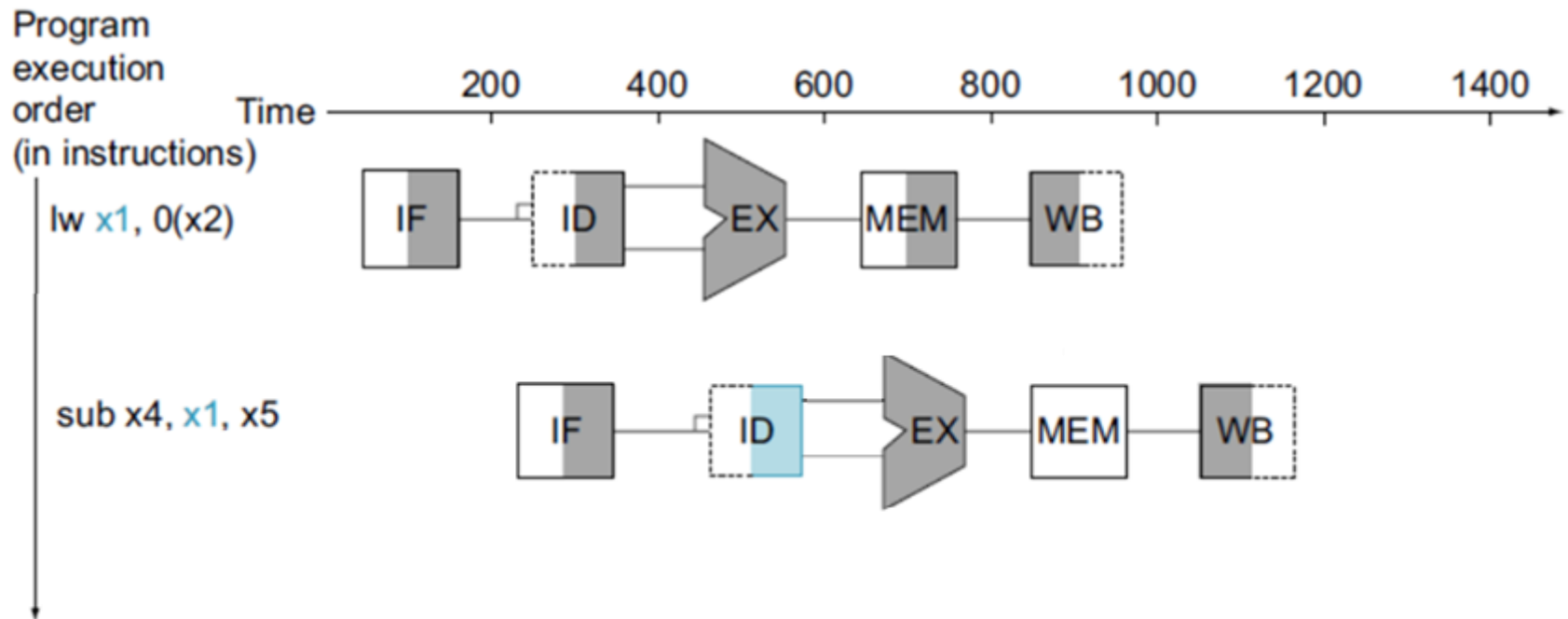
Forwarding (aka Bypassing)

- Use result as soon as it is computed
 - Requires extra connections in the datapath



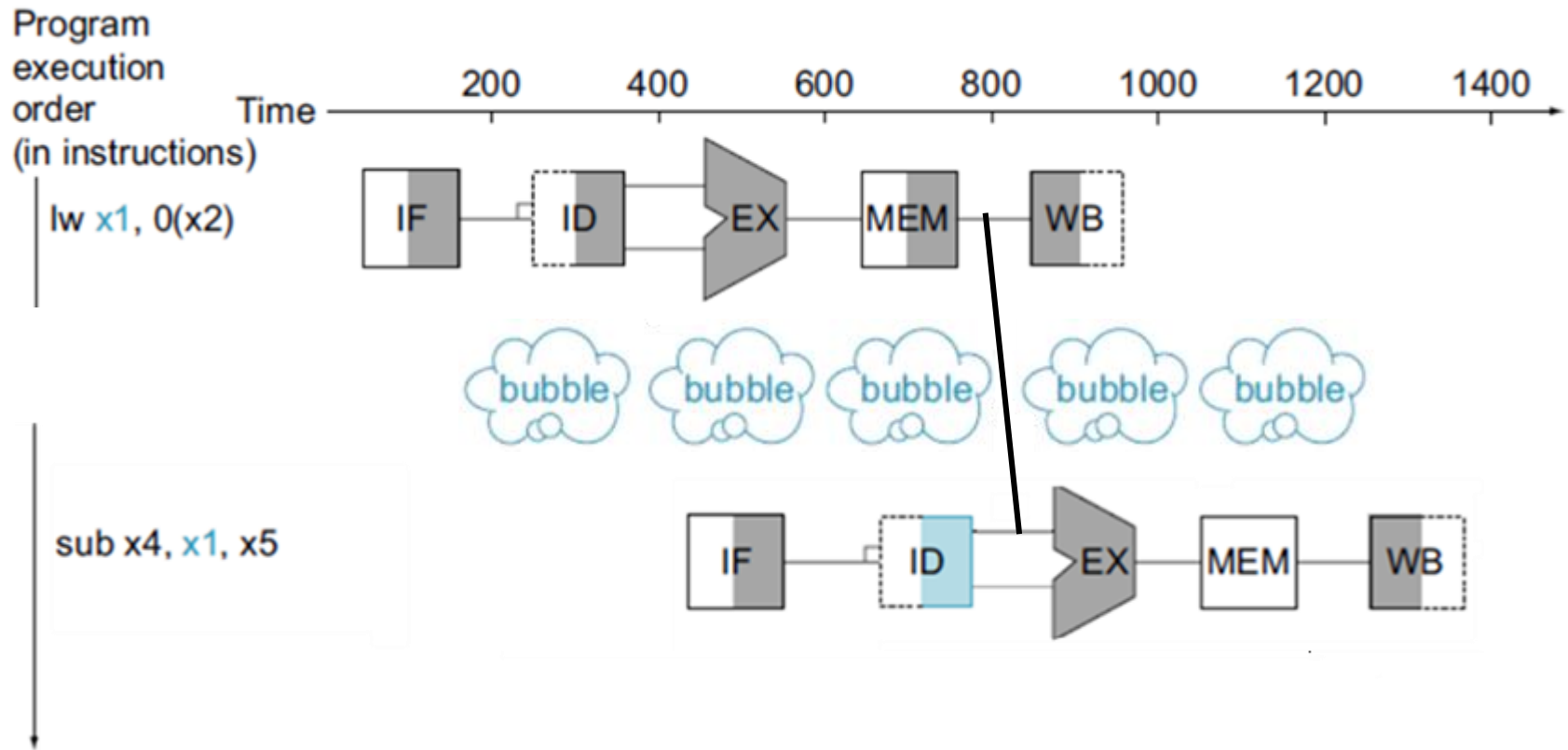
Load-Use Data Hazard

- Can't always avoid stalls by forwarding
 - If value not computed when needed
 - Need to stall! (Add a bubble)
 - Solution also possible via software re-ordering



Load-Use Data Hazard

- Can't always avoid stalls by forwarding



Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for $a = b + e; c = b + f;$
- Stalls?

<code>lw x1, 0(x31)</code>	<code>// Load b</code>
<code>lw x2, 4(x31)</code>	<code>// Load e</code>
<code>add x3, x1, x2</code>	<code>// b + e</code>
<code>sw x3, 12(x31)</code>	<code>// Store a</code>
<code>lw x4, 8(x31)</code>	<code>// Load f</code>
<code>add x5, x1, x4</code>	<code>// b + f</code>
<code>sw x5, 16(x31)</code>	<code>// Store c</code>

Total cycles ?

Code Scheduling to Avoid Stalls

- **Reorder code** to avoid use of load result in the next instruction
- C code for $a = b + e; c = b + f;$

```
lw    x1, 0(x31)
lw    x2, 4(x31)
lw    x4, 8(x31)
add   x3, x1, x2
sw    x3, 12(x31)
add   x5, x1, x4
sw    x5, 16(x31)
```

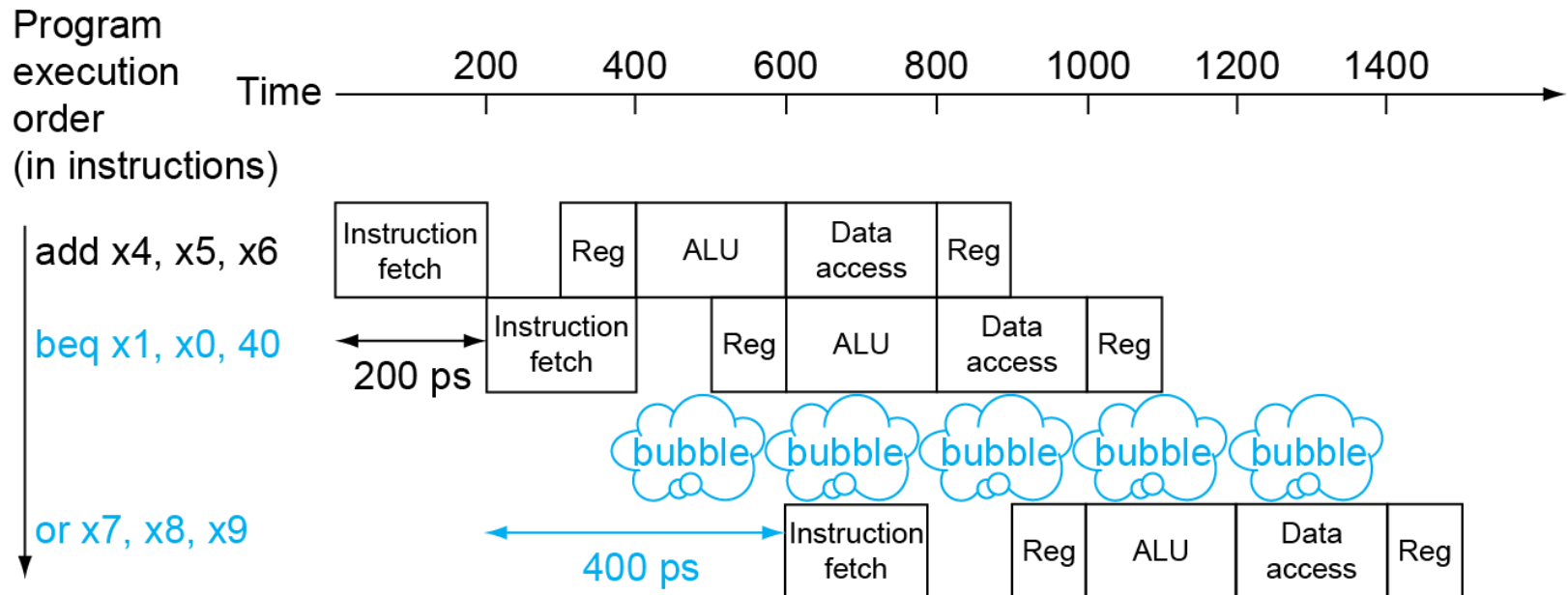
Total cycles

Control / Branch Hazards

- Branch determines flow of control
 - Fetching next instruction depends on branch outcome
 - Pipeline can't always know what the next instruction should be
- In RISC-V pipeline
 - Need to compare registers and compute target early in the pipeline
 - Add hardware to do it in ID stage

Stall on Branch

- Wait until branch outcome determined **before fetching next instruction**
 - Using **extra hardware**
- **Performance impact** (SPECint2006 with CPI = 1, 10% conditional branches, **impact?**)

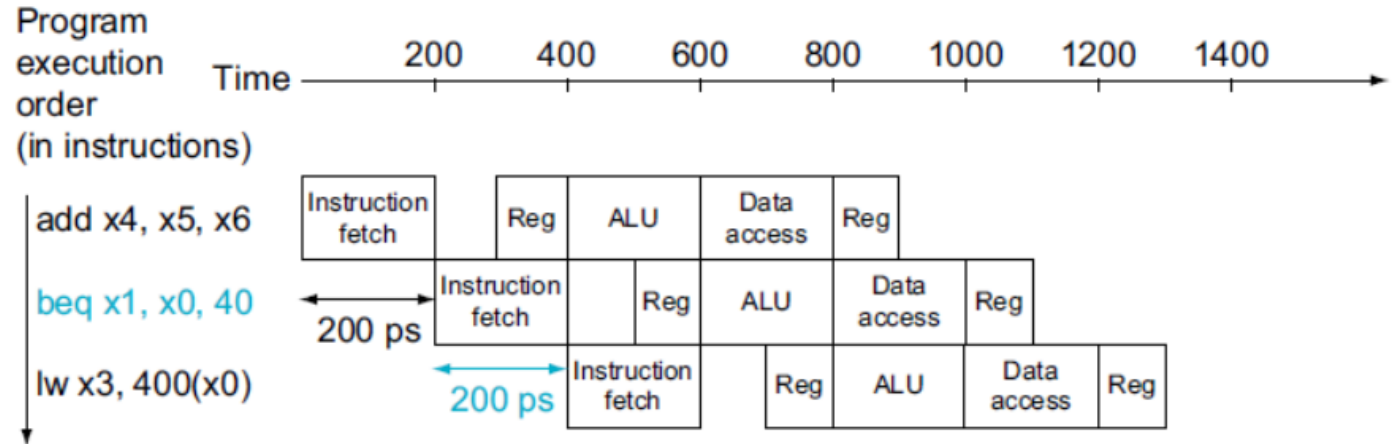


Branch Prediction

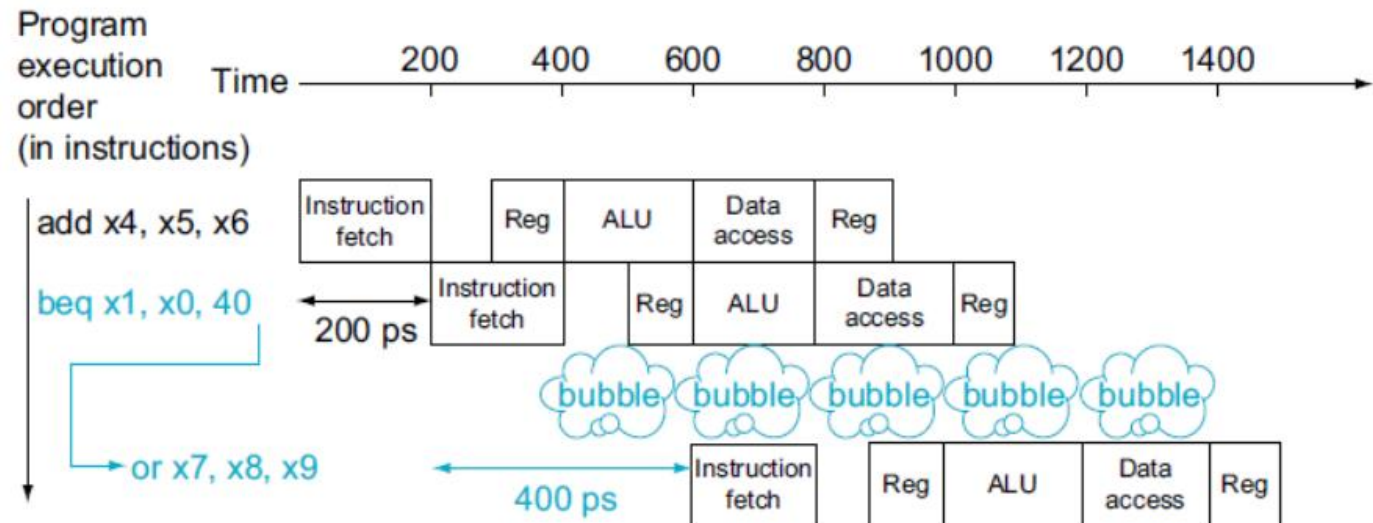
- Longer pipelines can't readily determine branch outcome early
 - Stall penalty becomes unacceptable
- Solution = Predict outcome of branch
 - Only stall if prediction is wrong
- In RISC-V pipeline
 - Can predict/assume branches not taken
 - Fetch instruction after branch, with no delay

Branch Prediction

Branch
not
taken
(default)



Branch
taken



More-Realistic Branch Prediction

- **Static branch prediction**
 - Based on **typical branch behavior**
 - Example: loop and if-statement branches
 - Predict **backward branches taken**
 - Predict forward branches not taken
- **Dynamic branch prediction**
 - Hardware measures **actual branch behavior**
 - e.g., **record recent history** of each branch
 - Assume future behavior will **continue the trend**
 - When wrong, stall while re-fetching, and update history

Pipeline Summary

- Pipelining improves performance by increasing instruction throughput
 - Executes multiple instructions in parallel
 - Each instruction has the same latency
- Subject to hazards
 - Structure, data, control
- Instruction set design affects complexity of pipeline implementation

Chapter Outline

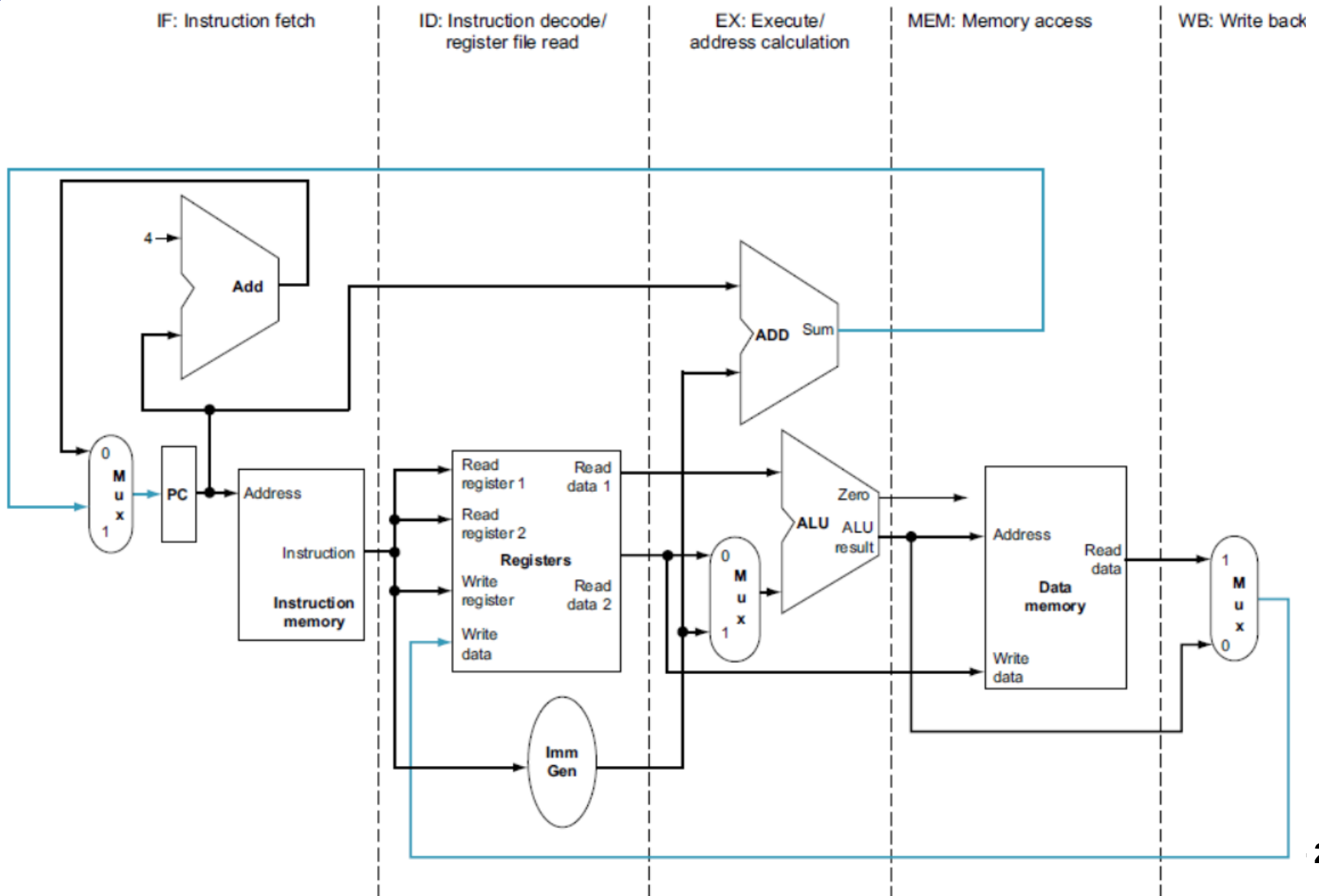
Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

RISC-V Pipelined Datapath

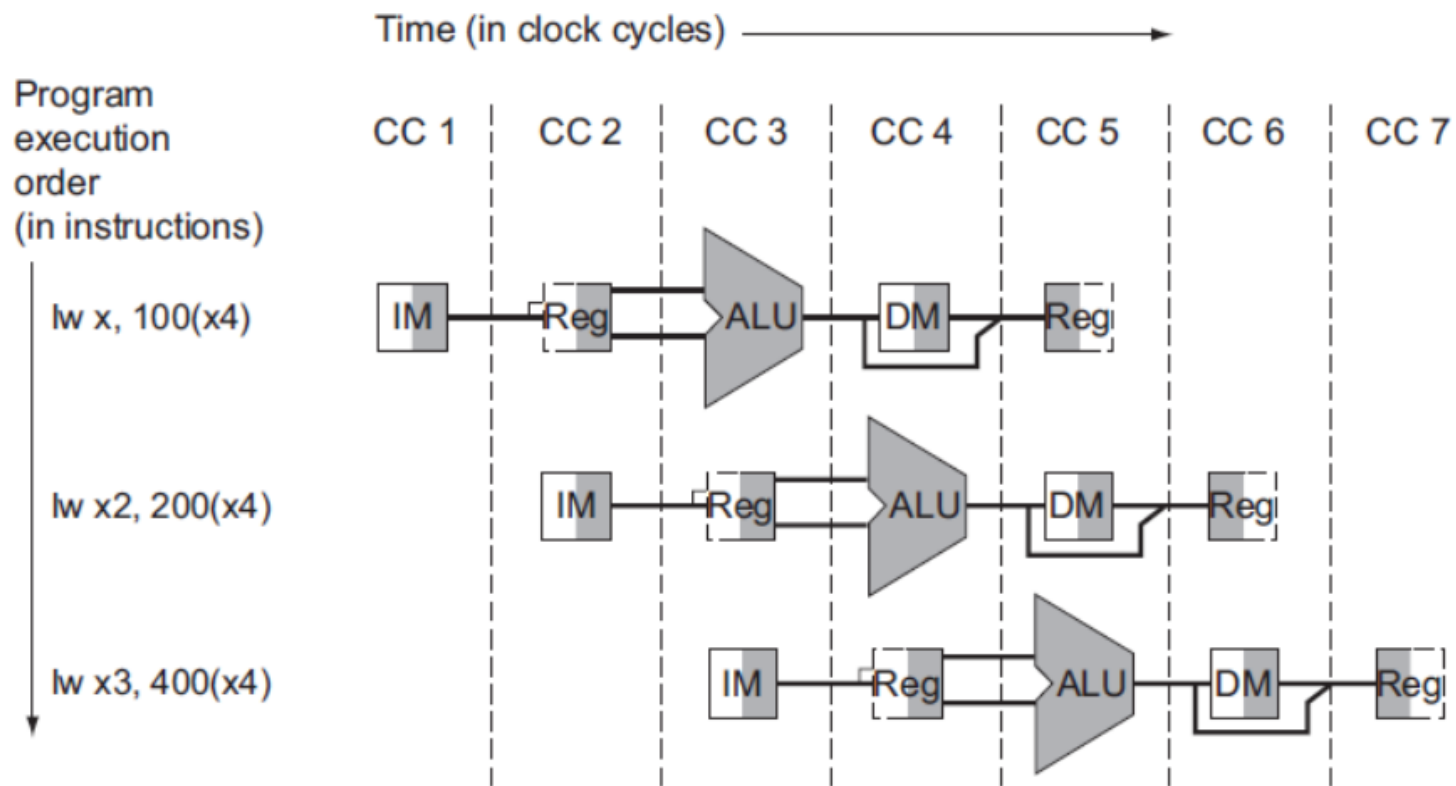
- We saw previously that instruction execution can be divided into 5 stages of pipeline
- How many instructions in parallel?
- RISC-V datapath can correspondingly be divided into 5 pieces/segments
- Data moves left-to-right in the datapath with two exceptions

RISC-V Single-cycle Pipelined Datapath



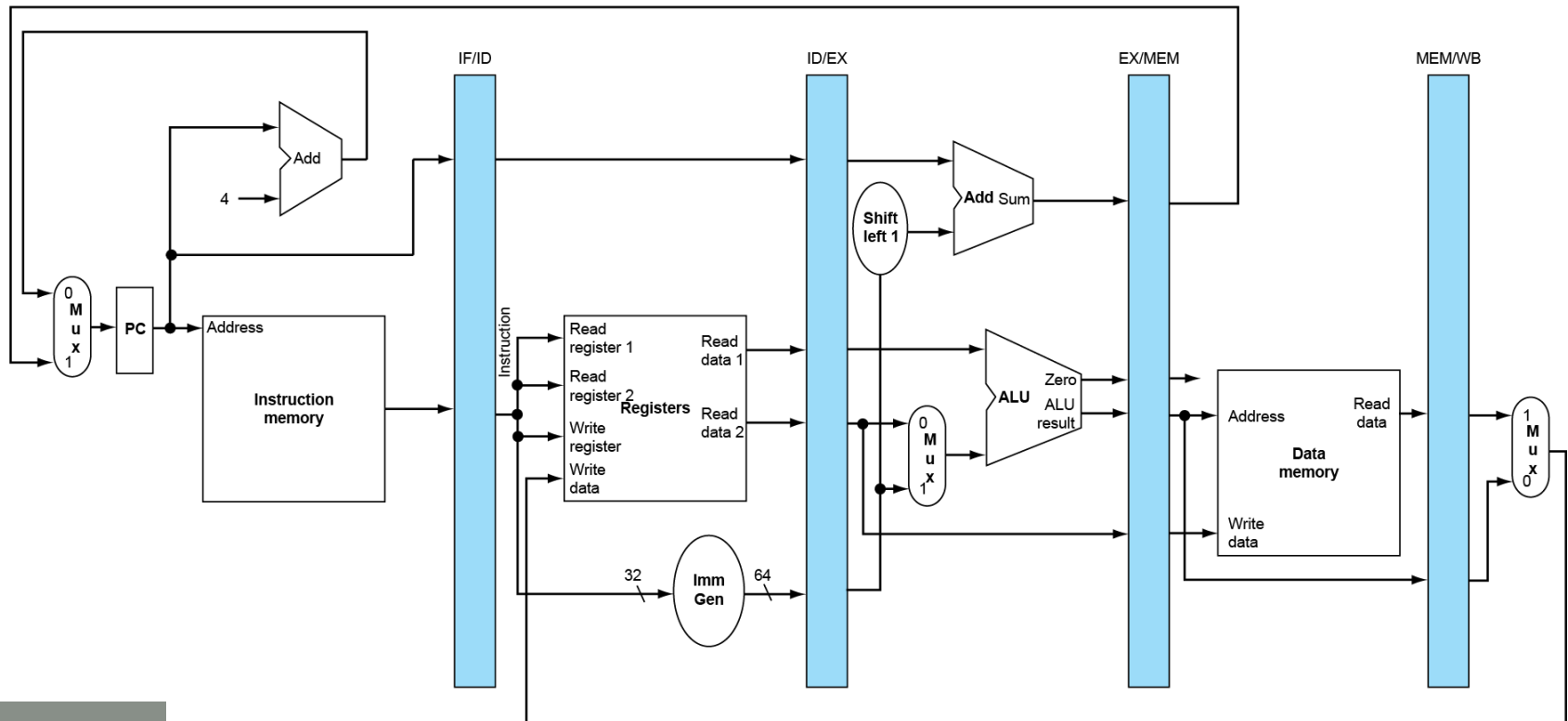
RISC-V Single-cycle Pipelined Datapath

- For ease of visualization, assume each instruction has its own dedicated datapath



Pipeline registers

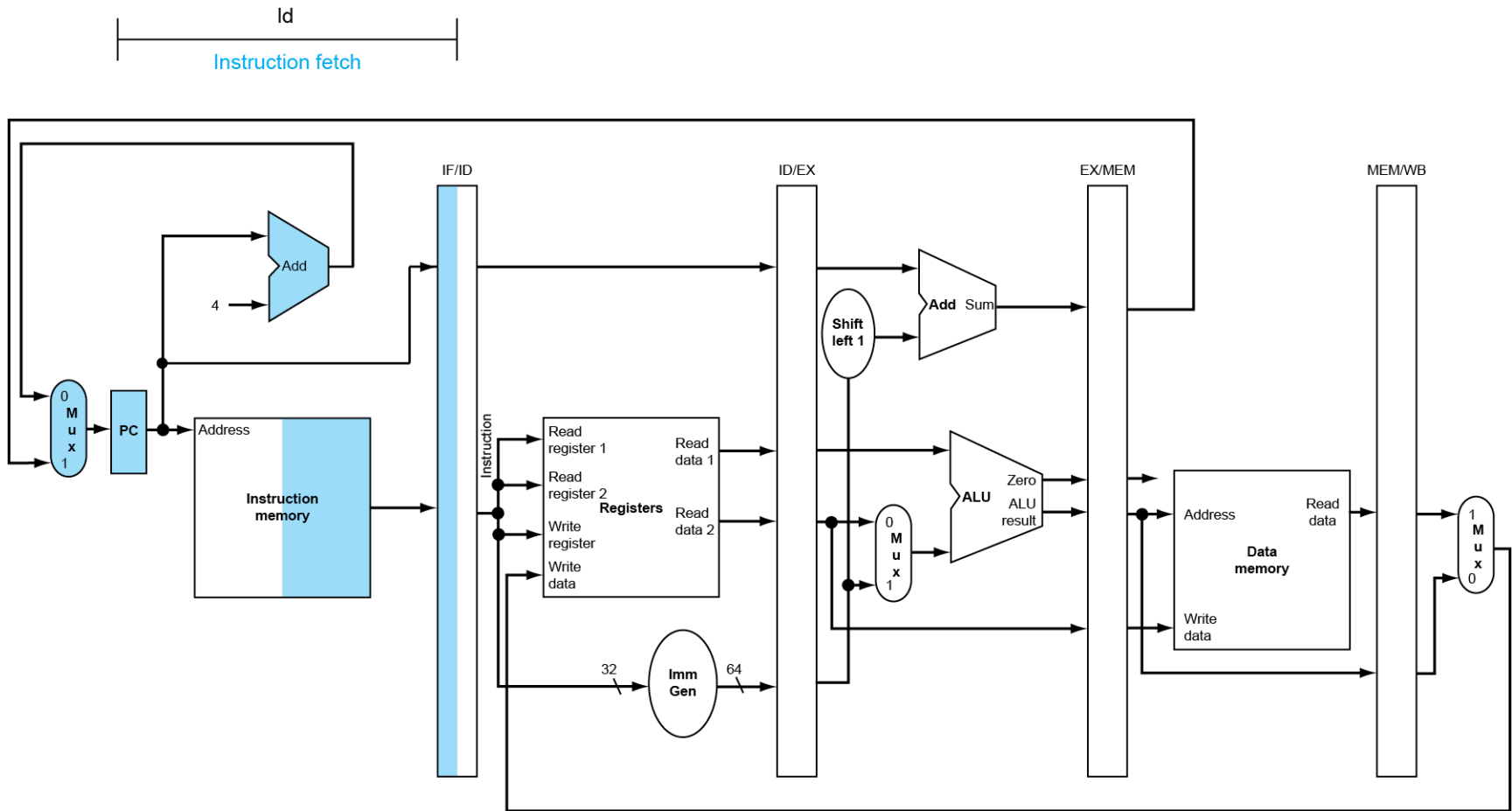
- In reality, a single datapath gets shared amongst instructions via use of registers
 - No pipeline register after the WB stage
 - Registers width?



Pipeline Operation

- Pipeline operation will be demonstrated via cycle-by-cycle flow of instructions through the pipelined datapath. This will be done using two types of figures:
 1. “Single-clock-cycle” pipeline figures
 - Shows pipeline usage in a single cycle
 - Highlight resources used
 2. “Multi-clock-cycle” figures
 - Graph of operation over multiple clock cycles
- We will first look at “single-clock-cycle” diagrams for load & store

IF for Load, Store, ...



ID for Load, Store, ...

